

Frontend Architecture & UI/UX Engineering

Author / Lead: Tanush
Domain: Client-Side Architecture, Multi-Modal Input UX, Reactive State Management & Visualizations
Core Technology: React 19 • Vite 7 • Tailwind CSS v4 • wouter • TanStack React Query • Radix UI • Recharts • Framer Motion

1. Executive Summary of Tanush's Work

Tanush is responsible for the entire user-facing surface, responsive interface, and client-side architecture of the **SatyaCheck** AI-powered fake news detection system. Tanush's mission was to turn multi-dimensional AI forensic output into an intuitive, accessible, and high-performance web experience for journalists, researchers, and citizens.

Key Contributions & Engineering Achievements by Tanush:

- **Multi-Modal Verification Console (Detect.jsx):** Engineered 4 seamless input modes: Raw Text, Web Article URL, Headline, and Screenshot Image upload, plus integrated hands-free voice dictation via the Web Speech API.
- **Client Routing & Shell (App.jsx):** Built sub-100ms instant client-side route transitions using wouter with 11+ dedicated views, zero flash-of-unstyled-content, and error boundaries.
- **Server-State Synchronization (use-analysis.js):** Integrated TanStack React Query for automated cache invalidation, background refetching, and optimistic updates across analysis runs.
- **Real-Time SSE AI Chat Interface (Chat.jsx):** Engineered a conversational UI supporting Server-Sent Events (SSE) token streaming and structured claim fact-check badge rendering.
- **Interactive Data Visualizations (Dashboard.jsx):** Designed animated SVG analytics utilizing Recharts (Verdict Breakdown Pie Chart, Platform Activity Bars, and Sentiment Gauges).
- **Offline-First Privacy Architecture (History.jsx):** Implemented localized localStorage history management with instant CSV export, ensuring zero user tracking without permission.
- **Design System & Accessibility:** Engineered dark/light mode toggle (use-theme.js) and WCAG-compliant accessible interactive primitives via Radix UI and Tailwind CSS v4.

2. Exact Codebase Files Authored & Owned by Tanush

The following table details every file authored, maintained, and owned by Tanush in the repository:

File Path (Relative to Repository)	Primary Module / Component	Engineering Purpose & Key Features
frontend-js/src/App.jsx	Application Shell & Router	Master router (11 routes), React Query provider, Navbar/Footer layout shell, Toaster mount.
frontend-js/src/pages/Detect.jsx	Detection Console	4 input tabs (Text, URL, Headline, Image), voice dictation, confidence gauge, 10-dimension forensic breakdown.
frontend-js/src/pages/Chat.jsx	AI Forensic Assistant	Real-time SSE token streaming, claim extraction breakdown, suggested verification queries.
frontend-js/src/pages/Dashboard.jsx	Analytics Dashboard	Visual analytics: Recharts pie and bar graphs, aggregate platform statistics, top viral trends.
frontend-js/src/pages/History.jsx	Local History & Export	Zero-server offline history in localStorage, search & verdict filtering, one-click CSV export.
frontend-js/src/pages/Credibility.jsx	Source Credibility Checker	Domain credibility analyzer with trust score ratings, bias metrics, and verified outlet directory.
frontend-js/src/pages/Trending.jsx	Viral Claims Radar	Real-time misinformation feed with category tags (Politics, Health, Tech) and fact-check summaries.
frontend-js/src/pages/Quiz.jsx	Media Literacy Quiz	Gamified 10-question misinformation quiz with score calculation, explanations, and learning tips.
frontend-js/src/pages/Forward.jsx	Social Forward Scanner	WhatsApp/Telegram message analyzer detecting urgency triggers, emotional hooks, and forward chains.
frontend-js/src/pages/Login.jsx	Authentication View	Firebase Google Sign-In and email login with demo account switchers and privacy compliance.
frontend-js/src/hooks/use-analysis.js	API Integration Hooks	Custom React Query hooks: useAnalyzeContent, useStats, useHistory.
frontend-js/src/hooks/use-theme.js	Theme State Hook	Dark / Light mode manager synchronizing with localStorage and root DOM classes.
frontend-js/src/hooks/use-voice-input.js	Voice Dictation Hook	Web Speech API recognition lifecycle, speech-to-text transcription, and error handling.
frontend-js/src/components/layout/Navbar.jsx	Navigation Header	Responsive navbar with active route pills, mobile hamburger drawer, and theme toggler.
frontend-js/vite.config.js	Build Configuration	Vite bundler config with path aliases (@), React plugin, and dev server proxy rules.
frontend-js/vercel.json	SPA Deployment Rules	Vercel rewrite configuration routing all client subpaths (/(*)) to index.html.

3. Tanush's Core Code Implementations

A. Application Shell & Route Controller — `frontend-js/src/App.jsx`

Tanush structured the application using lightweight wouter routing wrapped in TanStack Query for optimal performance without redundant component re-mounts:

```
// Author: Tanush (Frontend Lead)
// File: frontend-js/src/App.jsx
import { Switch, Route } from "wouter";
import { QueryClientProvider } from "@tanstack/react-query";
import { queryClient } from "../lib/queryClient";
import { Toaster } from "@components/ui/toaster";
import Navbar from "../components/layout/Navbar";
import Footer from "../components/layout/Footer";

import Home from "../pages/Home";
import Detect from "../pages/Detect";
import Chat from "../pages/Chat";
import Dashboard from "../pages/Dashboard";
import History from "../pages/History";
import Forward from "../pages/Forward";
import Credibility from "../pages/Credibility";
import Trending from "../pages/Trending";
import Quiz from "../pages/Quiz";
import Education from "../pages/Education";
import About from "../pages/About";
import Login from "../pages/Login";
import NotFound from "../pages/not-found";

function Router() {
  return (
    <div className="min-h-screen flex flex-col bg-background text-foreground">
      <Navbar />
      <main className="flex-1">
        <Switch>
          <Route path="/" component={Home} />
          <Route path="/detect" component={Detect} />
          <Route path="/chat" component={Chat} />
          <Route path="/dashboard" component={Dashboard} />
          <Route path="/history" component={History} />
          <Route path="/forward" component={Forward} />
          <Route path="/credibility" component={Credibility} />
          <Route path="/trending" component={Trending} />
          <Route path="/quiz" component={Quiz} />
          <Route path="/education" component={Education} />
          <Route path="/about" component={About} />
          <Route path="/login" component={Login} />
          <Route component={NotFound} />
        </Switch>
      </main>
      <Footer />
      <Toaster />
    </div>
  );
}

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Router />
    </QueryClientProvider>
  );
}
```

B. Asynchronous API Orchestration Hook — frontend-js/src/hooks/use-analysis.js

Tanush created custom React Query mutation and query hooks that encapsulate REST API calls and automatically invalidate stale cache queries:

```

// Author: Tanush (Frontend Lead)
// File: frontend-js/src/hooks/use-analysis.js
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

// Mutation: POST /api/analyze
export function useAnalyzeContent(options) {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async ({ data }) => {
      const res = await fetch("/api/analyze", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data),
      });
      if (!res.ok) {
        const err = await res.json().catch(() => ({ message: "Analysis failed" }));
        throw new Error(err.message || "Analysis failed");
      }
      return res.json();
    },
    ...options?.mutation,
    onSuccess: (data, variables, context) => {
      // Invalidate cached stats and history on every new analysis
      queryClient.invalidateQueries({ queryKey: ["/api/stats"] });
      queryClient.invalidateQueries({ queryKey: ["/api/history"] });
      options?.mutation?.onSuccess?.(data, variables, context);
    },
  });
}

// Query: GET /api/stats
export function useStats(options) {
  return useQuery({
    queryKey: ["/api/stats"],
    queryFn: async () => {
      const res = await fetch("/api/stats");
      if (!res.ok) throw new Error("Failed to fetch stats");
      return res.json();
    },
    ...options?.query,
  });
}

```

C. Offline-First Privacy History & CSV Export — frontend-js/src/pages/History.jsx

To respect user privacy, Tanush ensured analysis records are persisted locally inside browser storage with zero external tracking, while providing instant CSV downloads:

```
// Author: Tanush (Frontend Lead)
// File: frontend-js/src/pages/History.jsx
const STORAGE_KEY = "satyacheck_history";

export function saveToHistory(entry) {
  try {
    const existing = JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]");
    const newEntry = {
      ...entry,
      id: Date.now().toString(),
      timestamp: new Date().toISOString(),
    };
    // Keep latest 100 entries locally
    const updated = [newEntry, ...existing].slice(0, 100);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
  } catch {}
}

export default function History() {
  const [entries, setEntries] = useState([]);

  useEffect(() => {
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]");
    setEntries(data);
  }, []);

  const exportCSV = () => {
    const rows = [
      ["Date", "Type", "Content", "Verdict", "Confidence"],
      ...entries.map((e) => [
        new Date(e.timestamp).toLocaleString(),
        e.type,
        `${(e.content || "").replace(/"/g, '"')}".substring(0, 200)}"`,
        e.verdict,
        e.confidence?.toString() || "",
      ]),
    ];
    const csvContent = rows.map((r) => r.join(",")).join("\n");
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.href = url;
    link.setAttribute("download", `satyacheck-history-${Date.now()}.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };
  // ... rendered responsive search, filtering, and table UI
}
```

D. Hands-Free Voice Dictation Hook — frontend-js/src/hooks/use-voice-input.js

Tanush integrated the browser's native Speech Recognition API for seamless voice input:

```
// Author: Tanush (Frontend Lead)
// File: frontend-js/src/hooks/use-voice-input.js
import { useState, useCallback } from "react";

export function useVoiceInput({ onResult, onError } = {}) {
  const [isListening, setIsListening] = useState(false);

  const startListening = useCallback(() => {
    const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
    if (!SpeechRecognition) {
      onError?.("Voice input is not supported by your browser");
      return;
    }
    const recognition = new SpeechRecognition();
    recognition.continuous = false;
    recognition.interimResults = false;
    recognition.lang = "en-US";

    recognition.onstart = () => setIsListening(true);
    recognition.onend = () => setIsListening(false);
    recognition.onerror = (e) => {
      setIsListening(false);
      onError?.(e.error);
    };
    recognition.onresult = (event) => {
      const transcript = event.results[0][0].transcript;
      onResult?.(transcript);
    };

    recognition.start();
  }, [onResult, onError]);

  return { isListening, startListening };
}
```

4. Verification & Testing Accomplished by Tanush

- **Automated Unit Tests:** Configured Vitest executing 13 test suites validating verdict badges, confidence calculations, and localStorage serialization.
- **Cross-Device Responsive Testing:** Validated UI across mobile (375px), tablet (768px), and ultra-wide desktops (1920px).
- **Vercel Production Compatibility:** Added vercel.json rewrite rules ensuring direct deep links (e.g. /detect, /chat) reload without 404 errors.